Using TelecomRAG for Analysis of 5G MAC layer logs
Mid-Term Status

Dhanraj Shrivastav (CS24MTECH11004)

Project Overview (Review)

- Problem statement
  - Build RAG pipeline to perform detailed analysis of MAC layer logs using an LLM
- Key Objectives
  - Replicate prompt dependency finding of LLM5G-MAC.
  - Propose new scenarios causing MAC layer faults and collect relevant logs using OAI/ns3.
  - Evaluate whether LLMs are effective at providing detailed analyses of such faults
  - Evaluate whether RAG assists in eliminating prompt dependency

Methodology & Threat Model (Review)

- Proposed Solution Approach
  - Briefly show your high-level design diagram or approach from the proposal (Next slide)
- Threat Model (if applicable)
  - Not applicable.
- Key Assumptions
  - Logs must contain some key event an LLM can follow and trace its impact.

High Level Design Diagram

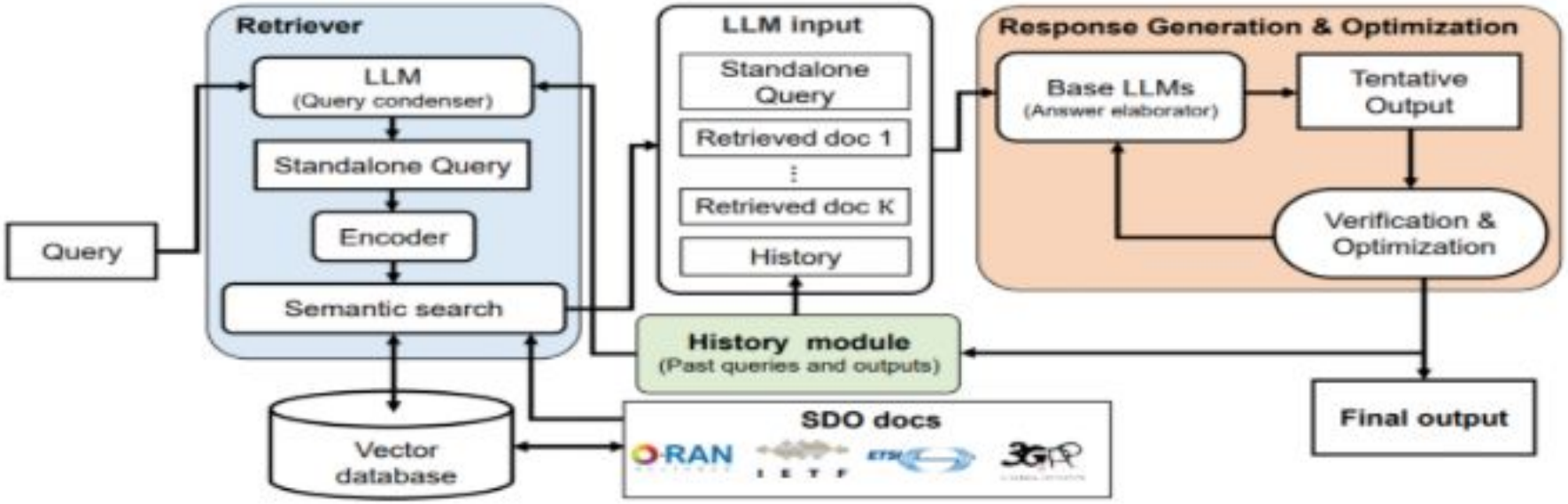


Figure 1: TelecomRAG: Architecture of the RAG-based Telecommunication Standards Assistant.

Mid-Term Deliverables (Status)

Promised Deliverables	Status (Completed/InProgress/Delayed)
Collect MAC logs. Replicate findings of LLM5G-MAC	Completed
Propose new scenario causing MAC layer faults using ns-3 or OAI.	Completed

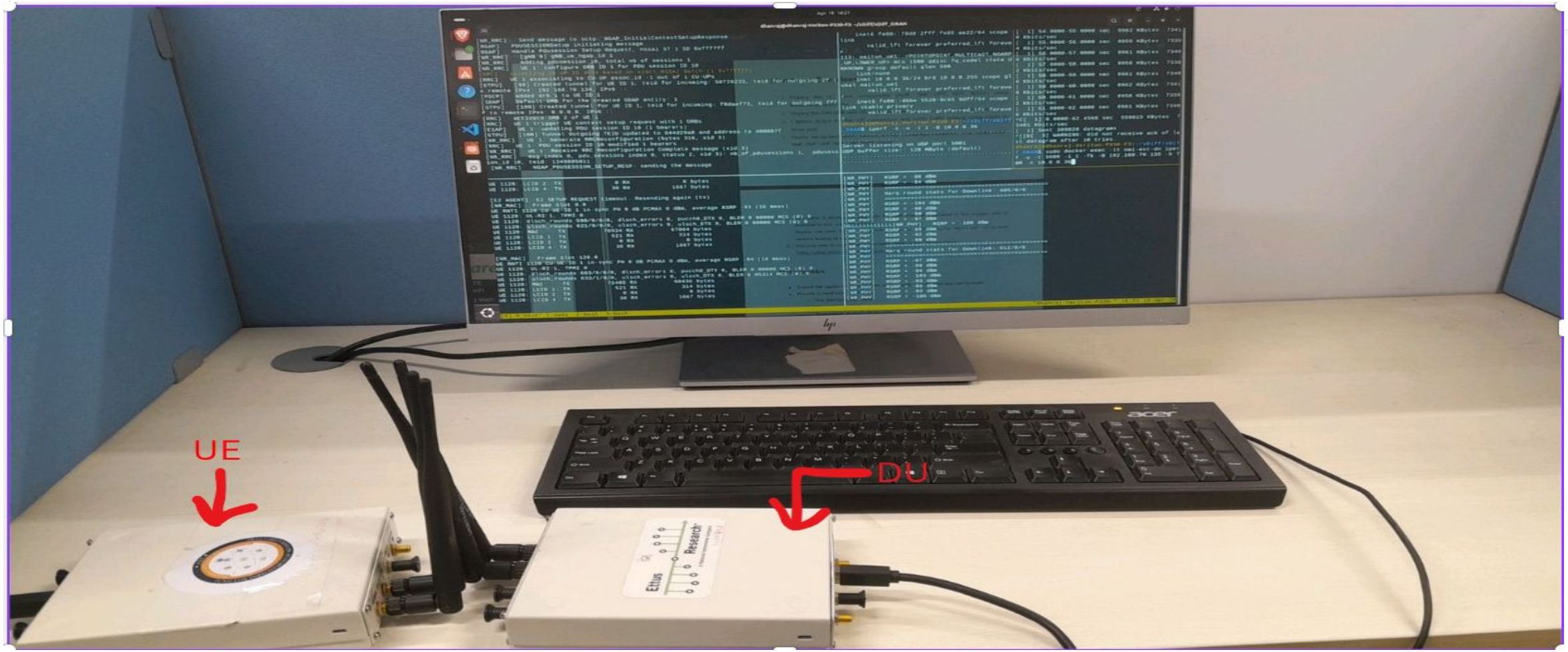
Work Completed & Implementation Details

- Hardware Setup (if applicable):
  - Used USRP UE and DU to collect logs to replicate paper findings.
  - Add a diagram or pic of your setup (see slide 8)
- Software & Tools:
  - OAI, ns-3 5G module (for additional HARQ exhaustion)
- Implementation:
  - (https://github.com/dhanraj-s/5GMACLogAnalysisRAG)

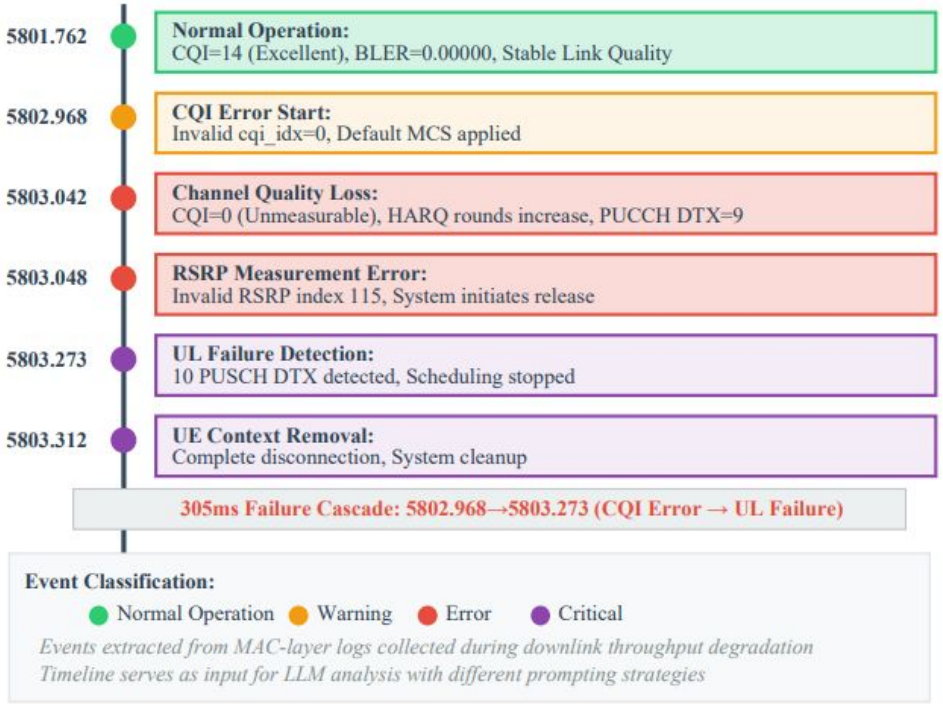
Experimental Setup

- Setup
  - Experiment-1: Two USRP devices (one UE, one DU). Moved apart.
  - Experiment-2: ns-3 HARQ exhaustion scenario.
    - 1 gNB, 1 UE
    - Very far (10km), low gNB transmit power, high UE noise figure
    - Remote Host sends UDP packet downlink every 1ms.
- Metrics used for Analysis
  - LLM should be able to perform root cause analysis. Producing a causal chain if a targeted prompt is given.
  - Above is the key observation of LLM5G-MAC.
- Baseline(s) for Comparison
  - ChatGPT-5 with generic prompt.

Picture of Setup



Preliminary Results - Paper Findings



Ground Truth

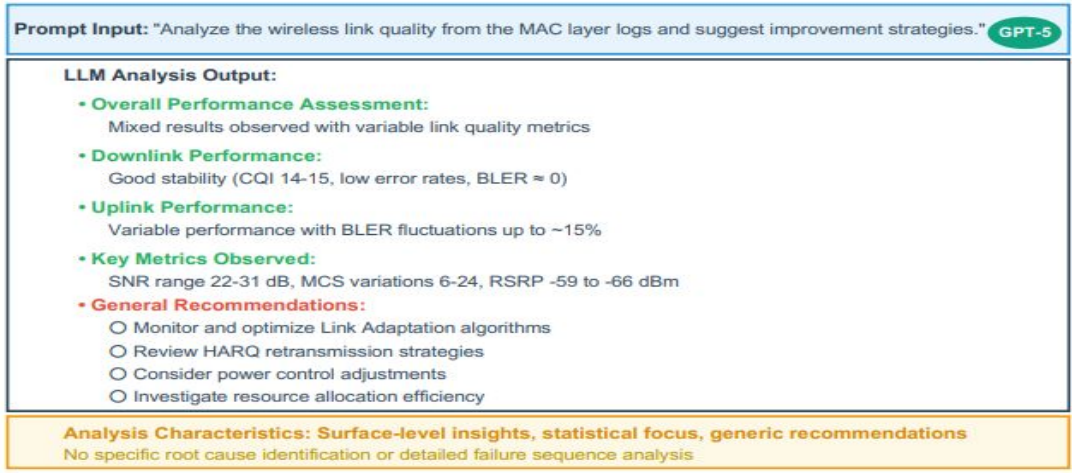


Figure 5: General prompt analysis results showing surface-level performance assessment without specific root cause identification.

General Prompt

Preliminary Results - Paper Findings (continued)

Targeted Prompt Input: "Analyze the CQI measurement error from the MAC layer logs and trace its impact on subsequent connection quality." GPT-5

LLM Comprehensive Analysis Output:

- **Root Cause Chain Identified:**
  - L2/L1 timing desynchronization → UCI corruption → CQI measurement failure
  - SFN/slot mismatch errors preceded invalid CQI index 0 at 5802.968077
- **Comprehensive Failure Timeline:**
  - ~5787.8: Multiple SFN/slot mismatch errors (L2↔L1 timing slip)
  - 5802.968: CQI decode breaks → "invalid cqi_idx 0, default to MCS 9"
  - 5803.042: CQI=0, PUCCH DTX spikes (5→9), dlsch_errors increase (1→2)
  - 5803.048: RSRP index 115 invalid (UCI corruption confirmation)
  - 5803.273: "UL Failure after 10 PUSCH DTX" → Connection termination
- **System Impact Analysis:**
  - Immediate spectral efficiency drop: DL MCS 20→9 (QPSK fallback)
  - Control channel degradation: PUCCH DTX spike indicates UCI unreliability
  - ~305ms total cascade from timing slip to complete disconnection
- **Targeted Technical Solutions:**
  - ○ Stabilize L2↔L1 timing: Fix NFAPI clocking and buffering issues
  - ○ Increase PUCCH target SNR: 100→120-140 (10→12-14 dB margin)
  - ○ Enhance UCI robustness: Consider CSI on PUSCH vs PUCCH
  - ○ Implement MCS smoothing: Prevent immediate fallback to MCS 9
  - ○ Add CQI/RSRP validity checks: Reject out-of-range indices

Key Analysis Features:

- Multi-layer root cause tracing
- Timing→UCI→CQI correlation
- Specific technical solutions
- 305ms cascade quantification

Analysis Characteristics: Deep root cause analysis, comprehensive timeline, systematic solutions

Multi-layer correlation: Timing→UCI→CQI→Control→Termination with technical fixes

- Targeted Prompt:
  - Tells the LLM a key event
  - Asks to trace its impact
- Key event: <u>CQI Measurement Error</u>

Preliminary Results - Replication

- Logs collected from Experiment-1 had causal chain (determined manually)
  - Invalid TA offset -> PUCCH DTX, invalid RSRP index (UCI corruption) -> increased BLER, dlsch_errors -> disconnection

Preliminary Results - Replication

Generic prompt: *Analyze the wireless link quality from all of the given MAC layer logs (all batches) and suggest improvement strategies.*

Response: (summarized)

1. moderate to weak RSRP
2. UL instability leading to failure
3. timing advance errors
4. control channel issues
5. UE removed

Targeted prompt: *Analyze the Invalid timing advance offset error from the MAC layer logs (all batches) and trace its impact on subsequent connection quality.*

Response: Causal Chain identified (see next slide)

Preliminary Results - Replication

```
Invalid TA offset
        ↓
UL time misalignment
        ↓
↑ UL BLER + ↑ HARQ retransmissions
        ↓
↑ PUCCH DTX (lost ACK/NACK)
        ↓
UL decoding errors (MAC PDU corruption, invalid LCID)
        ↓
gNB loses UL reliability
        ↓
UL Failure (PUSCH DTX threshold hit)
        ↓
Random Access triggered (TA reset attempt)
        ↓
Temporary recovery
        ↓
TA issue persists / reoccurs
        ↓
Repeated UL degradation
        ↓
RNTI release / connection breakdown
```

Preliminary Results - Experiment-2

- Setup details in slide 8.
- ns-3 logs are not as detailed and thorough as OAI logs
  - Used callbacks attached to trace sources to obtain OAI-like logs.

```
UE ffd4: LCID 4: TX              6 RX            138 bytes

 [0m[NR_MAC]   Frame.Slot 384.0
UE RNTI ffd4 CU-UE-ID 1 in-sync PH 0 dB PCMAX 0 dBm, average RSRP -92 (16 meas)
UE ffd4: UL-RI 1, TPMI 0
UE ffd4: dlsch_rounds 222/0/0/0, dlsch_errors 0, pucch0_DTX 0, BLER 0.00000 MCS (0) 9
UE ffd4: ulsch_rounds 233/0/0/0, ulsch_errors 0, ulsch_DTX 0, BLER 0.00000 MCS (0) 9
UE ffd4: MAC:    TX          27338 RX          23812 bytes
UE ffd4: LCID 1: TX            521 RX            314 bytes
UE ffd4: LCID 2: TX              0 RX              0 bytes
UE ffd4: LCID 4: TX              6 RX            138 bytes

 [0m[NR_MAC]   Frame.Slot 512.0
UE RNTI ffd4 CU-UE-ID 1 in-sync PH 0 dB PCMAX 0 dBm, average RSRP -94 (16 meas)
UE ffd4: UL-RI 1, TPMI 0
UE ffd4: dlsch_rounds 235/0/0/0, dlsch_errors 0, pucch0_DTX 0, BLER 0.00000 MCS (0) 9
UE ffd4: ulsch_rounds 246/0/0/0, ulsch_errors 0, ulsch_DTX 0, BLER 0.00000 MCS (0) 9
UE ffd4: MAC:    TX          28937 RX          25290 bytes
UE ffd4: LCID 1: TX            521 RX            314 bytes
UE ffd4: LCID 2: TX              0 RX              0 bytes
UE ffd4: LCID 4: TX              6 RX            138 bytes
```

(OAI)

```
[PHY  ][0.402536] DL CTRL SINR  | RNTI 1 | SINR -96.71 dB
[PHY  ][0.402536] DL TB SIZE    | RNTI 1 | TBS 136 B
[PHY  ][0.402964] DL DATA SINR  | RNTI 1 | SINR -96.71 dB
[PHY  ][0.402964] CQI FEEDBACK  | RNTI 1 | WB-CQI 0 | MCS 0 | RI 1
0.403s [MAC] NR HARQ NACK #1 for RNTI=1 ProcessID=15
[MAC  ][0.403000] DL HARQ       | RNTI 1 | PID 15 | NACK | Retx 0
[PHY  ][0.403000] SLOT DATA     | Frame 40 Sub 3 Slot 0 | Cell 1 BWP 0 | A
AvailSym 12
[PHY  ][0.403036] DL CTRL SINR  | RNTI 1 | SINR -96.71 dB
[PHY  ][0.403036] DL TB SIZE    | RNTI 1 | TBS 136 B
[PHY  ][0.403464] DL DATA SINR  | RNTI 1 | SINR -96.71 dB
[PHY  ][0.403464] CQI FEEDBACK  | RNTI 1 | WB-CQI 0 | MCS 0 | RI 1
0.4035s [MAC] NR HARQ NACK #2 for RNTI=1 ProcessID=14
[MAC  ][0.403500] DL HARQ       | RNTI 1 | PID 14 | NACK | Retx 0
[PHY  ][0.403500] SLOT DATA     | Frame 40 Sub 3 Slot 1 | Cell 1 BWP 0 | A
```

ns-3 with callbacks

Preliminary Results - Experiment-2 (contd)

- Key observation in HARQ exhaustion logs:
  - WB-CQI=0, SINR=-96.71dB -> abysmal channel -> HARQ processes fail -> base station eventually gives up (scheduling stall)

Preliminary Results - Experiment-2 (contd)

General Prompt: *Analyze the wireless link quality from all of the given MAC layer logs (all batches) and suggest improvement strategies.*

Response: (summarized)

1. link unusable in current state.
2. UE reports worst possible channel quality
3. TBS=136bytes (scheduler already minimizing TBS to improve reliability)

Targeted Prompt: *Analyze the CQI=0 error from the MAC layer logs and trace its impact on subsequent connection quality.*

Response: (causal chain identified, see next slide.)

Preliminary Results - Experiment-2 (contd)

```
Extreme SINR (~ -96 dB)
        ↓
CQI = 0
        ↓
MCS = 0, minimal TBS
        ↓
Decoding fails (BLER ~100%)
        ↓
HARQ NACKs accumulate
        ↓
HARQ processes exhausted
        ↓
MAC scheduling stall
        ↓
Throughput collapse + link failure
```

Live Demo

- (This slide is a placeholder to switch to your live demo)
- Even a short, 1-2 minute demo of your testbed or simulation output
  - (Live demo or play a recorded clip)
  - Experiment-1: https://drive.google.com/file/d/1kjCfcGxhwrQJdIzs-D2p7DZFM1Y5vLdK/view?usp=drive_link
  - Experiment-2: https://drive.google.com/file/d/1iA5UdVXQVtrtYcaMpVxhmuJ-JrNzOdF7/view?usp=drive_link

Challenges & Risks Encountered

- Original Risks:
  - Risk 1: Poor retrieval quality. (Mitigation: Query Translation)
  - Risk 2: Local open-source LLM too weak for technical reasoning (Mitigation: Use Gemini Free Tier or Groq Free API)
  - Risk 3: Logs do not contain failure events (Mitigation: Use USRPs. USRP connection is unstable will most definitely contain failure events).
  - Risk 4: Prompt dependency not eliminated through RAG (Mitigation: find out the bottleneck. Is it retrieval quality? Is it LLM reasoning?)
- Challenges Faced:
  - Very difficult to create failure scenarios (other than disconnect) using OAI and real USRPs.
  - Tried CPU contention (got skipped RX samples from USRP, no "key event" in MAC logs).
  - Tried modifying OAI code to skip HARQ processes randomly. SDU buffers full but no multi-layer cascade in MAC observed.
  - Logs for both uploaded on github.
  - ns-3 hooking callbacks to correct trace sources required reading a lot of documentation to find the correct Config Paths.
- Mitigation
  - Risk 3 → invalid TA failure event observed in disconnection case.

Updated Timelines & Next Steps - Timeline

- Week 1 (April 6-12): Collect MAC logs. Replicate findings of LLM-5GMAC (prompt dependency)
  - Completed
- Week 2 (April 13-19): Propose new scenarios that cause MAC layer faults. Eg: inject delays, increased scale causing scheduling faults, interference, etc. Collect logs for these scenarios using ns-3 or OAI 5G. (this week may involve debugging ns-3 and OAI issues).
  - Completed
- Week 3 (April 20-26): Obtain detailed analyses of these faults using LLMs. Evaluate general prompting against targeted prompting to assess LLM's ability to provide analyses.

  Build a TelecomRAG pipeline <u>or</u> modify an <u>existing implementation</u> based on OAI MAC layer logs, ns-3 documentation, 3GPP standards, etc.

  May have to build a new RAG to obtain results with custom document set.

  Existing: https://github.com/netop-team/Telco-RAG (will have to judge its feasibility; custom documents?)
  - In progress
- Week 4 (April 27-May 3): Test whether use of RAG helps eliminate the problem of prompt dependency. Qualitative analysis, citation rate and citation accuracy, https://deepeval.com/guides/guides-rag-evaluation. Prepare final report.
  - In progress

Note: This timeline is revised after feedback from Presentation-1. I presented this in my repeat presentation.

References

- GitHub Repo Link: https://github.com/dhanraj-s/5GMACLogAnalysisRAG
  - Clearly mark code snippets generated using LLMs with comment "<Name of LLM> Generated Code Snippet for TWiN Project"
- References:
  - LLM5G-MAC: https://dl.acm.org/doi/abs/10.1145/3737900.3770164
  - 5G-LENA: All TraceSources: https://cttc-lena.gitlab.io/nr/html/_trace_source_list.html
  - TelecomRAG: https://dl.acm.org/doi/10.1145/3711992.3711996

Declaration on LLM Usage

- List which LLMs you leveraged
  - Claude
- Include some sample prompts you used
  - "How would I build an OAI style logger in ns-3?"
  - "Here are the 5G-LENA Trace Sources from official docs along with their Config Paths…"
- How did you validate LLM generated scripts?
  - Confirming with documentation.
- How did you document your LLM generated code snippets?
  - Through comments in code.